ORGNZ-05: Bucket-Level Access Control

Series: ORGNZ | **Notebook:** 5 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Overview

Bucket-level access control provides a straightforward way to isolate data by team, application, or business unit. By granting permissions to specific buckets, you can ensure teams only access data relevant to their responsibilities.

Prerequisites

Requirement	Details
Dynatrace Account	Account-level administrative access
Permissions	IAM policy management permissions
Knowledge	Completed ORGNZ-04

Learning Objectives

By the end of this notebook, you will:
- Create IAM policies for bucket-level access
- Use bucket naming patterns in policies
- Implement team isolation using buckets
- Understand bucket permission best practices

Bucket Permission Fundamentals

Required Permissions

All bucket access policies must start with `storage:buckets:read`:

```
ALLOW storage:buckets:read WHERE &lt;condition&gt;;
ALLOW storage:logs:read;  // Then allow table access
```

Two-Step Pattern

Step	Purpose	Example
1. Bucket access	Define which buckets	`storage:buckets:read WHERE bucket-name = 'x'`
2. Table access	Define which data types	`storage:logs:read`, `storage:metrics:read`

Policy Examples

Example 1: Single Bucket Access

Grant a team access to their specific bucket:

```
{
  "name": "platform-team-logs-access",
  "description": "Platform team can access platform logs bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name = 'team_platform_logs'; ALLOW storage:logs:read;",
  "tags": ["team:platform"]
}
```

Example 2: Multiple Buckets with IN Operator

Grant access to a list of specific buckets:

```
{
  "name": "finance-multi-bucket-access",
  "description": "Finance team can access multiple buckets",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name IN
('finance_logs', 'finance_metrics', 'finance_audit'); ALLOW
storage:logs:read, storage:metrics:read;",
  "tags": ["team:finance"]
}
```

Example 3: Bucket Name Pattern with STARTSWITH

Grant access to all buckets matching a prefix:

```
{
  "name": "prod-infrastructure-access",
  "description": "Access to all production infrastructure buckets",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'prod_infra_'; ALLOW storage:logs:read, storage:metrics:read,
storage:spans:read;",
  "tags": ["env:production"]
}
```

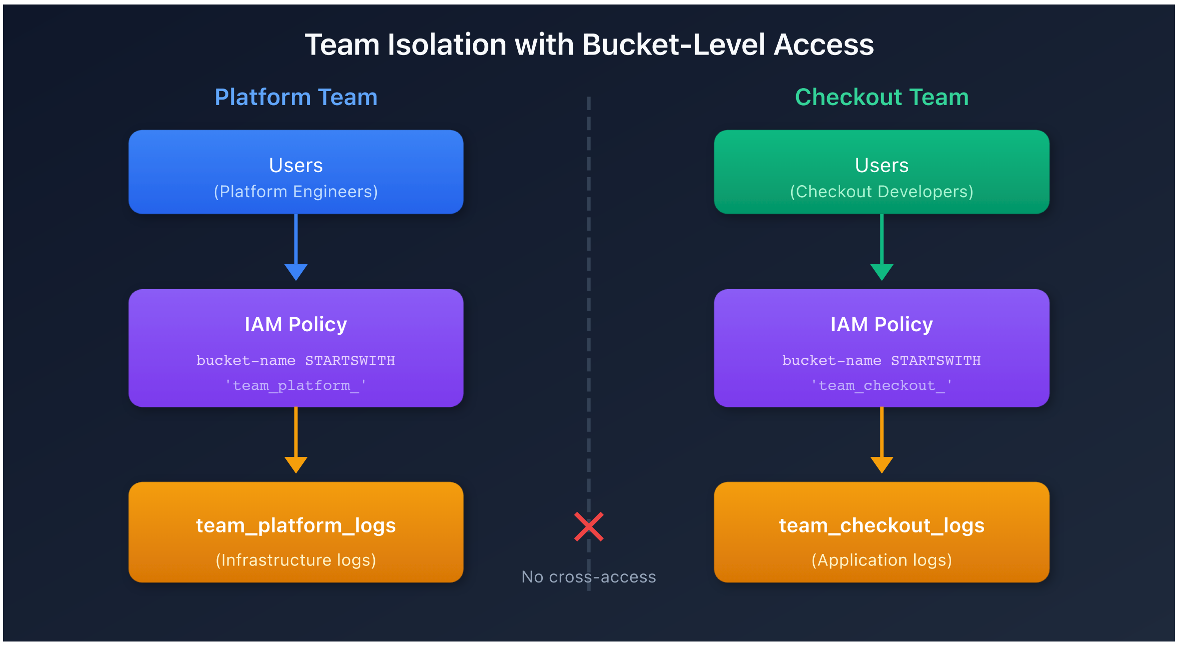
Example 4: Bucket Pattern with MATCH

Use pattern matching for complex bucket names:

```
{
  "name": "database-team-access",
  "description": "Database team can access any database-related bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
MATCH ('*-database-*'); ALLOW storage:logs:read;",
  "tags": ["team:database"]
}
```

Team Isolation Pattern

Architecture



Implementation

Step 1: Create team buckets

```
team_platform_logs
team_checkout_logs
team_payments_logs
```

Step 2: Route data via OpenPipeline

```
processors:
  - type: route
    rules:
      - condition: "host.group starts-with 'platform-'"
        destination: "team_platform_logs"
      - condition: "service.name contains 'checkout'"
        destination: "team_checkout_logs"
```

Step 3: Create IAM policies per team

```
// Platform team policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
"team_platform_";
ALLOW storage:logs:read;

// Checkout team policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
"team_checkout_";
ALLOW storage:logs:read;
```

Default Buckets Policy

Standard Default Access

For users who need access to all default buckets:

```
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "default_";
ALLOW storage:events:read, storage:logs:read, storage:metrics:read,
      storage:entities:read, storage:bizevents:read, storage:spans:read;
```

Default Plus Custom Buckets

```
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "default_";
ALLOW storage:buckets:read WHERE storage:bucket-name = "audit_logs_365d";
ALLOW storage:logs:read, storage:metrics:read, storage:spans:read;
```

Verifying Bucket Access

```
// List buckets you have access to
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days
| sort name asc
```

```
// Query data from specific bucket to verify access
fetch logs, bucket: "team_platform_logs"
| limit 10
```

```
// Check data distribution across accessible buckets
fetch logs
| summarize count = count(), by:{dt.system.bucket}
| sort count desc
```

Best Practices

Practice	Rationale
Use consistent bucket naming	Enables pattern-based policies
Assign policies to groups, not users	Easier management, consistent access
Document all policies	Audit trail and governance
Test policies with sample users	Prevent access issues
Use STARTSWITH for team prefixes	Future-proof for new buckets
Combine with record-level for scale	Bucket limits (80) may not suffice

When Bucket-Level Isn't Enough

Bucket-level access has limitations:

Limitation	Alternative
80 bucket limit	Use security context for finer granularity
Shared data needs	Use record-level permissions
Dynamic team membership	Use security context mapped to tags
Field-level masking	Use field-level permissions

See **ORGNZ-06** and **ORGNZ-07** for advanced permission patterns.

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-06**: Security Context

References

- Permissions in Grail
- IAM policy reference

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.